Tech Spec: Wikipedia Election Scraper (Apify Actor)

Version: 0.1

Creator: Nel Barthelemy

Status: Draft — v1 scope defined, not yet built

Last Updated: 2026-08-19

1. Overview

A standalone Apify Actor that looks up a U.S. Senate race by state and election cycle, and returns the current candidate list (name, party, status, short bio) sourced live from Wikipedia. Sold on the Apify Store as a paid scraper, targeting developers, researchers, and other applications that need normalized election-candidate data without building their own Wikipedia-parsing pipeline.

Relationship to the Political Transparency App: this reuses the Wikipedia heading-parse pipeline designed and live-tested for that project's Phase 1 nominee-resolution feature (see `tech-spec.md` §3, §6), but is a **separate, standalone product** — no shared database, no FEC cross-referencing, no shared deployment. It can ship before that app's Phase 1 is finished.

Key divergence from the Political Transparency App: that app permanently excludes party affiliation from all UI and payloads as a core product principle. This scraper does **not** carry that constraint — its customers are developers who may have legitimate reasons to want party data, and Wikipedia's own structure (party-labeled headings) makes it nearly free to include. `party` is a first-class output field here. This is a deliberate, explicit divergence, not an oversight — worth remembering if code or docs are ever shared between the two projects.

2. Goals (v0.1)

- Given a state, office, and election cycle, return the current candidate list for that race
- Source data live from Wikipedia (MediaWiki Action API), not a static/stale copy
- Distinguish real, current candidates (`declared` / `runoff_pending` / `nominee`) from primary losers, withdrawals, and other dropped statuses — same filtering logic as the main app
- Include `party` per candidate (see divergence note above), with true independents represented as `party: null`, not a fabricated value
- Cheap enough per-query (via caching) to be viable at Apify's pay-per-event pricing

3. Scope

In scope for v0.1:

- Query pattern: `{ state, office, cycle }` structured input (e.g. `state: "Michigan"` , `office: "senate"` , `cycle: 2026`) — not free-text natural-language parsing
- U.S. Senate races only
- Current election cycle only (2026) — title patterns for past cycles are unverified

- Regular and special elections (confirmed live for the 2026 FL/OH specials)
- Runoff handling (`runoff_pending` status, `runoff_date` field)

Explicitly out of scope for v0.1 (candidates for later versions, not designed yet):

- `"2026 elections in [state]"` — all-offices-in-a-state lookup. Wikipedia has no single clean structured page per state covering every office; likely needs a per-office-type loop or a category/list-page parse, neither of which has been tested.
- `"[candidate name]"` — name-based search. Different extraction path (search-and-disambiguate rather than direct title construction), not yet designed.
- Non-Senate offices (House, Governor, state legislature, local)
- FEC cross-referencing / candidate ID matching — this product is Wikipedia-only by design

4. Data Source

MediaWiki Action API (`en.wikipedia.org/w/api.php`) — free, public, no API key. Same two endpoints already live-verified against real 2026 Senate race data this session:

- `action=query&prop=extracts&explaintext=1&titles={title}` — primary lookup. Article text is heading-structured (`== {Party} primary ==` → `=== Candidates ===` → `==== Nominee/Declared/Advanced to runoff/... ====`) and machine-parseable.
- `action=query&list=search&srsearch={query}` — fallback when the constructed title doesn't resolve directly (confirmed necessary for some title variants).

Licensing: CC BY-SA 4.0. Commercial use is permitted. Extracted *facts* (candidate name, status, party) can be sold freely; if any raw article text is ever returned verbatim (e.g. a bio sentence), the output must carry attribution (`source_url` , `source_license`) — already built into the output schema below, not an afterthought.

Etiquette: requests must set a descriptive `User-Agent` identifying this product, per Wikipedia's API usage expectations — same practice used throughout this session's testing.

5. Input Schema

```
{
  "state": "Michigan",
  "office": "senate",
  "cycle": 2026
}
```

`state` accepts full names or 2-letter codes, normalized via a state-code ↔ full-name crosswalk (same lookup already flagged as needed infrastructure in the main app's spec, §12).

6. Pipeline

1. Normalize `state` to its full name.

2. Check a maintained per-cycle config (e.g. `{ 2026: ["FL", "OH"] }`) to determine whether this state's race is a special election, which changes the title template.
3. Construct the Wikipedia title deterministically:
   - Regular: `"{cycle} United States Senate election in {state}"`
   - Special: `"{cycle} United States Senate special election in {state}"`
4. Fetch via `prop=extracts`. If the title 404s, fall back to `list=search` and flag the result as `title_matched_via: "search_fallback"` so a low-confidence match is distinguishable from a direct hit.
5. Heading-parse the article text: split on `==` / `===` / `====` boundaries, walk into each party's `=== Candidates ===` subsection, and classify each listed candidate by their status heading.
6. Keep only candidates under `Declared`, `Advanced to runoff`, or `Nominee` headings. Drop everyone else (`Declined`, `Withdrawn`, `Did not file`, `Not on ballot`, `Disqualified`, `Eliminated in primary`, `Ineligible`) — same rule as the main app.
7. Record each kept candidate's party from the heading it was found under. For the `Independents` subsection specifically, set `party: null` rather than inventing one.
8. Extract `primary_date` (and `runoff_date`, if applicable) from the intro/background text. Two known sentence patterns exist — a single clean sentence for non-runoff states, a longer compound sentence for runoff states (confirmed live for Oklahoma) — extraction must handle both, not just the simpler one.
9. Cache the parsed result, keyed on `(state, office, cycle)`, with a TTL that expires once that race's own recorded primary/runoff date has passed. This is v0.1 scope, not a later optimization — it keeps Wikipedia request volume reasonable and keeps marginal cost per paid Apify run close to zero.
10. Return the structured result (§7).

7. Output Schema

```
{
  "query": { "state": "Michigan", "office": "senate", "cycle": 2026 },
  "source_url": "https://en.wikipedia.org/wiki/2026_United_States_Senate_election_in_Michigan",
  "source_license": "CC BY-SA 4.0",
  "title_matched_via": "direct",
  "primary_date": "2026-08-04",
  "runoff_date": null,
  "candidates": [
    { "name": "Abdul El-Sayed", "party": "Democratic", "status": "nominee", "bio": "Former Wayne
County health director..." },
    { "name": "Mike Rogers", "party": "Republican", "status": "nominee", "bio": "Former U.S.
Representative..." }
  ],
  "parse_confidence": "high",
  "last_updated": "2026-08-19T00:00:00Z"
}
```

`status` values: `declared` | `runoff_pending` | `nominee`. `parse_confidence` is `high` for a direct title match with cleanly-headed sections, `needs_review` when the search fallback was used or the heading structure didn't match the expected pattern cleanly.

8. Known Risks & Limitations

Carried over from the main app's research, but with a higher bar here since a parsing failure is now a paying customer's broken result, not an internal admin-reviewed data point:

- **Heading structure and status vocabulary aren't perfectly consistent across articles.** Confirmed live: New Hampshire's Republican section had a "Did not file" subsection missing its `====` markup entirely (appeared as bare text). The parser needs a fallback for malformed/missing headings, not pure heading-boundary parsing.
- **Disambiguation risk**: `"{state} Senate election"` (state legislature) vs `"United States Senate election in {state}"` (federal) are different pages with similar names — confirmed live for Michigan and Florida. Title construction must always use the precise federal phrasing.
- **Runoff-state date sentences use different, less predictable phrasing** than non-runoff states — only spot-checked against one example (Oklahoma) so far. Needs testing against a couple more runoff states (Texas, Louisiana) before this is trusted broadly.
- **Special-election detection is manually maintained**, not automatic — the per-cycle config (§6, step 2) needs a human to update it each cycle, since Wikipedia's data doesn't expose "this is a special election" as a queryable flag on its own.
- **Only spot-checked, not exhaustively tested** — three regular-election articles and one runoff article were verified this session. Before charging money per query, worth testing against a wider sample of the full candidate set (all 35 2026 Senate races) to catch parser edge cases before customers do.

9. Monetization

Sold via the Apify Store. Two pricing models are available to Actor creators:

- **Pay-per-event** — a fixed price for a defined event (e.g. "one race returned").
- **Pay-per-usage** — no creator-set price; users pay Apify directly for compute/proxy/storage.

Pay-per-event is the better fit here, given the low marginal compute cost once caching (§6, step 9) is in place. **Apify's exact revenue-share cut was not found in their public pricing docs as of this session** — needs direct confirmation from Apify before finalizing a price point.

10. Open Questions

- Exact per-query price point — blocked on confirming Apify's revenue-share cut.
- Whether/when to build query patterns 2 (all elections in a state) and 3 (candidate name search) as later versions.
- Whether historical-cycle support (pre-2026) is worth adding, and whether the title construction pattern even holds for older cycles — untested.